Overview of Weight initialization

Subject: Weight initialization

1.

Further reading Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron (2016). "8.4 Parameter Initialization Strategies". Deep learning. Adaptive computation and machine learning. Cambridge, Mass: The MIT press. ISBN 978-0-262-03561-3. Narkhede, Meenal V.; Bartakke, Prashant P.; Sutaone, Mukul S. (June 28, 2021). "A review on weight initialization strategies for neural networks". Artificial Intelligence Review. 55 (1). Springer Science and Business Media LLC: 291–322. doi:10.1007/s10462-021-10033-z. ISSN 0269-2821.

2.

Constant initialization We discuss the main methods of initialization in the context of a multilayer perceptron (MLP). Specific strategies for initializing other network architectures are discussed in later sections. For an MLP, there are only two kinds of trainable parameters, called weights and biases. Each layer l {\displaystyle l} contains a weight matrix W (l) ∈ R n l − 1 × n l {\displaystyle W^{(l)}\in \mathbb {R} ^{n_{l-1}\times n_{l}}} and a bias vector b (l) ∈ R n l {\displaystyle b^{(l)}\in \mathbb {R} ^{n_{l}}} , where n l {\displaystyle n_{l}} is the number of neurons in that layer. A weight initialization method is an algorithm for setting the initial values for W (l) , b (l) {\displaystyle W^{(l)},b^{(l)}} for each layer l {\displaystyle l} . The simplest form is zero initialization: W (l) = 0 , b (l) = 0 {\displaystyle W^{(l)}=0,b^{(l)}=0} Zero initialization is usually used for initializing biases, but it is not used for initializing weights, as it leads to symmetry in the network, causing all neurons to learn the same features. In this page, we assume b = 0 {\displaystyle b=0} unless otherwise stated. Recurrent neural networks typically use activation functions with bounded range, such as sigmoid and tanh, since unbounded activation may cause exploding values. (Le, Jaitly, Hinton, 2015) suggested initializing weights in the recurrent parts of the network to identity and zero bias, similar to the idea of residual connections and LSTM with no forget gate. In most cases, the biases are initialized to zero, though some situations can use a nonzero initialization. For example, in multiplicative units, such as the forget gate of LSTM, the bias can be initialized to 1 to allow good gradient signal through the gate. For neurons with ReLU activation, one can initialize the bias to a small positive value like 0.1, so that the gradient is likely nonzero at initialization, avoiding the dying ReLU problem.

3.

LeCun initialization LeCun initialization, popularized in (LeCun et al., 1998), is designed to preserve the variance of neural activations during the forward pass. It samples each entry in W (l) {\displaystyle W^{(l)}} independently from a distribution with mean 0 and variance 1 / n l − 1 {\displaystyle 1/n_{l-1}} . For example, if the distribution is a continuous uniform distribution, then the distribution is U (± 3 / n l − 1) {\displaystyle {\mathcal {U}}(\pm {\sqrt {3/n_{l-1}}})} .

4.

Fixup initialization In 2015, the introduction of residual connections allowed very deep neural networks to be trained, much deeper than the ~20 layers of the previous state of the art (such as the VGG-19). Residual connections gave rise to their own weight initialization problems and strategies.

These are sometimes called "normalization-free" methods, since using residual connection could stabilize the training of a deep neural network so much that normalizations become unnecessary. Fixup initialization is designed specifically for networks with residual connections and without batch normalization, as follows:

5.

In deep learning, weight initialization or parameter initialization describes the initial step in creating a neural network. A neural network contains trainable parameters that are modified during training: weight initialization is the pre-training step of assigning initial values to these parameters. The choice of weight initialization method affects the speed of convergence, the scale of neural activation within the network, the scale of gradient signals during backpropagation, and the quality of the final model. Proper initialization is necessary for avoiding issues such as vanishing and exploding gradients and activation function saturation. Note that even though this article is titled "weight initialization", both weights and biases are used in a neural network as trainable parameters, so this article describes how both of these are initialized. Similarly, trainable parameters in convolutional neural networks (CNNs) are called kernels and biases, and this article also describes these.

6.

Others Instead of initializing all weights with random values on the order of O (1 / n) {\displaystyle O(1/{\sqrt {n}})} , sparse initialization initialized only a small subset of the weights with larger random values, and the other weights zero, so that the total variance is still on the order of O (1) {\displaystyle O(1)} . Random walk initialization was designed for MLP so that during backpropagation, the L2 norm of gradient at each layer performs an unbiased random walk as one moves from the last layer to the first. Looks linear initialization was designed to allow the neural network to behave like a deep linear network at initialization, since W R e L U (x) − W R e L U (− x) = W x {\displaystyle W\;\mathrm {ReLU} (x)-W\;\mathrm {ReLU} (-x)=Wx} . It initializes a matrix W {\displaystyle W} of shape R n 2 × m {\displaystyle \mathbb {R} ^{{\frac {n}{2}}\times m}} by any method, such as orthogonal initialization, then let the R n × m {\displaystyle \mathbb {R} ^{n\times m}} weight matrix to be the concatenation of W , − W {\displaystyle W,-W} .

7.

He initialization As Glorot initialization performs poorly for ReLU activation, He initialization (or Kaiming initialization) was proposed by Kaiming He et al. for networks with ReLU activation. It samples each entry in W (l) {\displaystyle W^{(l)}} from N (0 , 2 / n l − 1) {\displaystyle {\mathcal {N}}(0,2/n_{l-1})} .

8.

Glorot initialization Glorot initialization (or Xavier initialization) was proposed by Xavier Glorot and Yoshua Bengio. It was designed as a compromise between two goals: to preserve activation variance during the forward pass and to preserve gradient variance during the backward pass. For uniform initialization, it samples each entry in W (l) {\displaystyle W^{(l)}} independently and identically from U (± 6 / (n l + 1 + n l − 1)) {\displaystyle {\mathcal {U}}(\pm {\sqrt {6/(n_{l+1}+n_{l-1})}})} . In the context, n l − 1 {\displaystyle n_{l-1}} is also called the "fan-in", and n l + 1 {\displaystyle n_{l+1}} the "fan-out". When the fan-in and fan-out are equal, then Glorot initialization is the same as LeCun initialization.